

로지스틱 회귀 적용

로지스틱 회귀 실험

- 실습 코드: [링크](#)
- 데이터셋: Iris 데이터
- 학습/시험 데이터: x_train/x_test
- 학습/시험 데이터 라벨: y_train/y_test → (0,1,2)
- 데이터 로더, 범주형 데이터 변환, 데이터 분할
 - KNN 실습과 동일
- 로지스틱 회귀 API
 - LogisticRegression 클래스 호출 $C=1/\lambda$ 로 규제화의 정도를 조절
 - C값이 클수록 규제화의 강도가 줄어듦

```
[5] 1 # Logistic regression
    2 from sklearn.linear_model import LogisticRegression
    3 Logit = LogisticRegression(C=200, random_state=11) # C = 1/λ. 디폴트: L2, Auto.
    4 l_1=Logit.fit(X_train_std, y_train)
    5 y_train_pred = Logit.predict(X_train_std)
    6 y_test_pred = Logit.predict(X_test_std)
    7
```

로지스틱 회귀 실험

- 로지스틱 회귀 평가
 - 학습데이터 정밀도(accuracy), 시험데이터 정밀도, 정확도 행렬(confusion matrix)

```
[6] 1 # Accuracy score
    2 from sklearn.metrics import accuracy_score
    3 print(accuracy_score(y_train,y_train_pred))
    4 print(accuracy_score(y_test,y_test_pred))
    5
```

```
0.9809523809523809
1.0
```

```
1 # Confusion matrix
2 from sklearn.metrics import confusion_matrix
3 print(confusion_matrix(y_test, y_test_pred)) # Confusion matrix
```

```
[[15  0  0]
 [ 0 15  0]
 [ 0  0 15]]
```

로지스틱 회귀 실험

- 실습코드: [링크](#)
- 데이터셋: 와인 데이터
- 학습/시험 데이터: $x_{\text{train}}/x_{\text{test}}$ (13개의 특성변수)
- 학습/시험 데이터 라벨: $y_{\text{train}}/y_{\text{test}}$ → (1,2,3) 로 이미 범주형으로 저장되어 있음
- 다양한 규제강도에 따른 실험
 - L_1, L_2 규제화
 - $C=1/\lambda$ 로 규제화
 - 테스트 데이터의 라벨을 알 수 없을 경우, 학습데이터의 일부를 검증데이터(validation data)로 구성하여 테스트
- 다양한 규제강도에 따른 초정계수 실험
 - 규제강도가 클수록 추정된 계수들의 절대값이 작아짐
 - L_1 규제화의 경우 규제강도가 클수록 계수에 0이 많아짐. 즉, 계수에 대응하는 특성변수를 제거하는 역할을 담당함

로지스틱 회귀 실험

■ 데이터 로더

```
[2] 1 # 데이터 불러오기. y값은 이미 범주형으로 되어있음.  
2 dat_wine=pd.read_csv('http://archive.ics.uci.edu/ml/machine-learning-databases/'  
3                       'wine/wine.data',header=None)  
4 dat_wine.head()  
5 dat_wine.columns = ['class label', 'alcohol', 'malic acid', 'ash',  
6                     'alcalinity of ash', 'magnesium', 'total phenols',  
7                     'flavanoids', 'nonflavanoid phenols',  
8                     'proanthocyanins', 'color intensity', 'hue',  
9                     'OD208', 'proline'] # Column names  
10 print('class label:', np.unique(dat_wine['class label'])) # Class 출력  
11 dat_wine.head()
```

```
class label: [1 2 3]
```

■ 학습/테스트 데이터 나누기

```
1 # 전체 data를 training set과 test set으로 split  
2 from sklearn.model_selection import train_test_split  
3 X, y = dat_wine.iloc[:,1:].values, dat_wine.iloc[:,0].values  
4 X_train, X_test, y_train,y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
```

로지스틱 회귀 실험

▪ L1, L2 규제화에 따른 실험



```
1 # Logistic Regression with L2 or L1 Regularization
2 from sklearn.linear_model import LogisticRegression
3 lr2_10 = LogisticRegression(penalty='l2', C=10.0, solver='saga') # L2 with C(=1/λ)=10
4 lr2_1 = LogisticRegression(penalty='l2', C=1.0, solver='saga') # L2 with C(=1/λ)=1
5 lr2_0_1 = LogisticRegression(penalty='l2', C=0.1, solver='saga') # L2 with C(=1/λ)=0.1
6
7 lr1_10 = LogisticRegression(penalty='l1', C=10.0, solver='saga') # L1 with C(=1/λ)=10
8 lr1_1 = LogisticRegression(penalty='l1', C=1.0, solver='saga') # L1 with C(=1/λ)=1
9 lr1_0_1 = LogisticRegression(penalty='l1', C=0.1, solver='saga') # L1 with C(=1/λ)=0.1
```

▪ L1, L2 규제화에 따른 정밀도



```
1 # 규제화 방법(L2 or L1)과 규제강도(λ)를 바꿔가며 accuracy score 계산
2 lr2_10.fit(X_train_std, y_train)
3 print('Training accuracy with L2 and λ=0.1:', lr2_10.score(X_train_std, y_train))
4 print('Test accuracy with L2 and λ=0.1:', lr2_10.score(X_test_std, y_test))
_
```

```
☞ Training accuracy with L2 and λ=0.1: 1.0
   Test accuracy with L2 and λ=0.1: 0.9814814814814815
```

로지스틱 회귀 실험

■ 규제화 강도에 따른 특성 변수 제거 관련

```
1 # L2 규제의 규제강도(C=1/λ)를 바꿔가며 계수 추정치 관찰
2 print(lr2_10.intercept_)
3 print(lr2_1.intercept_)
4 print(lr2_0_1.intercept_)
5
6 print(lr2_10.coef_)
7 print(lr2_1.coef_)
8 print(lr2_0_1.coef_)
```

```
[ 0.32296362  0.60033615 -0.92329977]
[ 0.28173282  0.6024029 -0.88413572]
[ 0.0683881  0.45688086 -0.52526895]
[[ 1.25664178  0.15899529  0.39926374 -1.53634982  0.08473759  0.37407767
  0.83876311 -0.28751864  0.09268499  0.12775152  0.0722625  0.97188895
  1.39116593]
[-1.5372395 -0.43915291 -1.23984712  1.21218732 -0.32703465 -0.51670074
  0.85895303  0.40866438  0.39442514 -1.36535608  1.14060554  0.02219384
 -1.75612335]
[ 0.28059772  0.28015763  0.84058338  0.32416249  0.24229707  0.14262307
 -1.69771614 -0.12114574 -0.48711013  1.23760457 -1.21286804 -0.99408279
  0.36495742]]
[[ 0.75495729  0.06165881  0.2336697 -0.8925231  0.02649841  0.29464787
  0.56064124 -0.2071806  0.13401678  0.12719203  0.10165733  0.61737663
  0.90976587]
[-0.98657135 -0.32327905 -0.65176965  0.66792906 -0.22933211 -0.20753188
  0.43824097  0.19874428  0.24373934 -0.78043161  0.63697798  0.08558965
 -1.03461549]
[ 0.23161406  0.26162025  0.41809995  0.22459404  0.2028337 -0.08711598
 -0.99888221  0.00843632 -0.37775612  0.65323959 -0.73863531 -0.70296628
  0.12484963]]
[[ 0.41030119 -0.03148562  0.13676704 -0.41134759  0.05383263  0.22360478
  0.31670971 -0.15968597  0.11370031  0.07036472  0.1110665  0.30981116
  0.51691919]
[-0.5426495 -0.20155646 -0.25667025  0.28071006 -0.14835806 -0.0406011
  0.12453008  0.0829087  0.10087435 -0.44571802  0.27319166  0.09645505
 -0.51870207]
[ 0.13234831  0.23304208  0.1199032  0.13063753  0.09452543 -0.18300368
 -0.44123979  0.07677727 -0.21457466  0.3753533 -0.38425816 -0.40626621
  0.00178288]]
```

```
1 # L1 규제의 규제강도(C=1/λ)를 바꿔가며 계수 추정치 관찰
2 print(lr1_10.intercept_)
3 print(lr1_1.intercept_)
4 print(lr1_0_1.intercept_)
5
6 print(lr1_10.coef_)
7 print(lr1_1.coef_)
8 print(lr1_0_1.coef_)
```

```
[ 0.39334184  0.5958625 -0.98920434]
[ 0.28462985  0.54413385 -0.8287637 ]
[ 0.05085504  0.30693808 -0.35779311]
[[ 1.19997383e+00  0.00000000e+00  1.54580407e-01 -1.57710818e+00
  0.00000000e+00  2.84110640e-01  8.25653391e-01  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  8.79401443e-01
  1.62309433e+00]
[-1.78064452e+00 -4.56713816e-01 -1.47383462e+00  1.05802427e+00
 -2.92893382e-01 -4.06457188e-01  7.01037258e-01  4.65557053e-01
  2.08045179e-01 -1.56217122e+00  1.10248967e+00  0.00000000e+00
 -1.93798201e+00]
[ 5.39392997e-02  1.17354860e-01  7.93799029e-01  0.00000000e+00
  1.77579266e-01  0.00000000e+00 -2.03734189e+00 -6.11569248e-06
 -3.41178760e-01  1.12646922e+00 -1.21297461e+00 -9.54584609e-01
  0.00000000e+00]]
[[ 0.0313239  0. -1.17721748  0. 0.
  0.02148411  0. 0. 0. 0. 0.62105
  0.97646011]
[-1.58450657 -0.1446383 -0.77389205  0.04388529 -0.0731721  0.
  0. 0.12895574  0. -0.98296559  0.23622794  0.
 -1.21587376]
[ 0. 0. 0. 0. 0.
 -2.08083573  0. -0.04410569  0.27113817 -0.80443491 -0.65929035
  0. ]]
[[ 0. 0. 0. -0.04184623  0. 0.
  0.23299536  0. 0. 0. 0.
  0.84042285]
[-0.8348383  0. 0. 0. 0.
  0. 0. 0. -0.4233421  0. 0.
 -0.20651752]
[ 0. 0. 0. 0. 0.
 -0.60150089  0. 0. 0.10503944 -0.3520872 -0.521087
  0. 0. ]]
```